MiniMind Pretrain 预训练：从原理、公式到源码逐行讲解

> *适用场景：大语言模型 / Transformer / 生成式 AI 课程*
> *对应仓库：用户提供的 MiniMind 源码*
> *重点文件：*
>
> - `trainer/train_pretrain.py`
> - `dataset/lm_dataset.py`
> - `model/model_minimind.py`
> - `trainer/trainer_utils.py`
> - `model/tokenizer.json`
> - `model/tokenizer_config.json`

---

0. 这节课到底要讲明白什么？

这一节课的目标不是只告诉学生“运行 `python train_pretrain.py` 就可以预训练”，而是要把下面这条链路真正讲透：

```
原始文本
    ↓
Tokenizer
    ↓
Token IDs
    ↓
添加 BOS / EOS
    ↓
Padding
    ↓
构造 input_ids / labels
    ↓
Embedding
    ↓
8 层 Decoder-only Transformer
    ↓
LM Head
    ↓
每个位置得到一个 6400 维 logits
    ↓
Shift 一位
    ↓
Cross Entropy
    ↓
反向传播
    ↓
AdamW 更新参数
    ↓
模型逐渐学会"根据前文预测下一个 token"
```

MiniMind 的预训练本质可以浓缩成一句话：

> ***给模型看大量文本，然后让它不断做"下一个 token 是什么？"这道题。***

但真正重要的是理解：

1. 为什么"预测下一个 token"可以学到语言、知识和一定程度的推理能力？
2. 一个句子究竟会产生多少个训练样本？
3. `input_ids` 和 `labels` 为什么几乎一模一样？
4. causal mask 为什么必不可少？
5. logits、softmax、cross entropy 在数学上是什么关系？
6. loss 是如何从一句话一路反传到几十万个甚至几千万个参数的？

7. MiniMind 代码中的每一部分分别对应哪一个数学概念？

---

1. 什么是 Pretraining？

1.1 从“训练一个分类器”说起

传统监督学习通常是：

$$ x \rightarrow y $$

例如：

> 输入：这部电影太好看了
> 标签：正面

模型的目标是：

$$ P(y \mid x) $$

但语言模型面对的问题不同。

我们没有必要人工给互联网中的每一句话标标签。

文本本身就包含监督信号。

例如：

> 北京是中国的首都

可以自动构造出：

> 输入：北京
> 目标：是
>
> 输入：北京是
> 目标：中国
>
> 输入：北京是中国
> 目标：的
>
> 输入：北京是中国的
> 目标：首都

因此，大规模文本可以自动变成训练数据。

这叫：

> ***Self-Supervised Learning，自监督学习。***

这里不需要人手工标注“正确答案”。

正确答案就是文本中的下一个 token。

---

2. 为什么叫“预训练”？

现代 LLM 常见训练链路是：

```
随机初始化模型
        ↓
  Pretraining
        ↓
  Base Model
        ↓
  SFT
        ↓
  Instruct / Chat Model
        ↓
  DPO / PPO / GRPO 等 Alignment
```

预训练阶段解决的是：

> ***让模型学会语言世界本身。***

包括但不限于：

- 词语之间的搭配关系
- 语法
- 上下文依赖
- 常识
- 世界知识
- 代码模式
- 数学表达模式
- 文档结构
- 一定程度的推理模式

SFT 才主要解决：

> *“用户问我问题时，我应该以什么形式回答？”*

所以：

Pretrain 更像“读书”

模型大量阅读：

> 新闻
>
> 百科
>
> 书籍
>
> 代码
>
> 论文
>
> 网页
>
> 对话
>
> ……

SFT 更像“教它怎么回答问题”

例如：

```
user: 什么是 Transformer？
  assistant: Transformer 是一种……
```

这是两个不同阶段。

---

3. Decoder-only LLM 的核心概率模型

MiniMind 是 Decoder-only Causal Language Model。

假设一句话经过 tokenizer 后得到：

$$ x*1*,*x*2,\ldots,x_T $$

语言模型试图学习整个序列的概率：

$$ P(x*1*,*x*2,\ldots,x_T) $$

根据概率链式法则：

$$ P(x*1*,*x*2,\ldots,x_T) = P(x_1) P(x*2*|*mid* x1) P(x*3*|*mid* x1,x_2) \cdots P(x*T*|*mid* x1,\ldots,x_{T-1}) $$

可以写成：

$$ \boxed{ P(x_{1:T}) = \prod_{t=1}^{T} P(x*t*|*mid* x{

其中：

$$x{1,\ldots,x_{t-1}) $$

这就是自回归语言模型（Autoregressive Language Model）。

---

4. Pretrain 的训练目标

如果模型希望正确文本的概率尽可能大，那么目标是：

$$ \max_\theta \prod_{t=1}^{T} P\theta(xt\mid x_{

因为连乘不好优化，对数之后：

$$ \max_\theta \sum_{t=1}^{T} \log P\theta(xt\mid x_{

机器学习通常写成最小化 loss：

$$ \boxed{ \mathcal{L} = -\sum_{t=1}^{T} \log P\theta(xt\mid x_{

如果取平均：

$$ \boxed{ \mathcal{L} = -\frac{1}{N} \sum_{t=1}^{N} \log P\theta(yt\mid x_{\leq t}) } $$

这就是 Negative Log-Likelihood。

对于词表分类问题，它正好等价于 Cross Entropy。

---

5. 一句话是如何变成很多道训练题的？

为了直观起见，假设 tokenizer 把：

> 我爱人工智能

切成：

> 我 | 爱 | 人工智能

再加特殊 token：

> <BOS> | 我 | 爱 | 人工智能 | <EOS>

那么模型实际上同时做四道题：

模型看到的前文	应预测
`<BOS>`	`我`
`<BOS> 我`	`爱`
`<BOS> 我 爱`	`人工智能`
`<BOS> 我 爱 人工智能`	`<EOS>`

注意：

> Transformer 并不是把这四道题真的循环跑四遍。

由于 causal mask 的存在，一次 forward 就可以并行计算所有位置。

这正是 Transformer 训练非常高效的原因之一。

---

6. MiniMind 中最关键的“Shift 一位”

MiniMind `PretrainDataset` 返回：

```
input_ids = [...]
  labels = input_ids.clone()
```

很多初学者看到这里会产生一个问题：

> input 和 label 完全一样，那模型不是直接抄答案了吗？

答案是：

> **没有。真正的错位是在模型 forward 中完成的。**

MiniMind：

```
x, y = logits[..., :-1, :].contiguous(), labels[..., 1:].contiguous()
  loss = F.cross_entropy(
      x.view(-1, x.size(-1)),
      y.view(-1),
      ignore_index=-100
  )
```

假设：

```
input_ids:
  [BOS, 我, 爱, AI, EOS]

  labels:
  [BOS, 我, 爱, AI, EOS]
```

模型产生：

```
logits:
  位置0预测 ...
  位置1预测 ...
  位置2预测 ...
  位置3预测 ...
  位置4预测 ...
```

代码取：

```
logits[..., :-1, :]
```

即：

```
位置0
  位置1
  位置2
  位置3
```

而 label 取：

```
labels[..., 1:]
```

即：

```
我
  爱
  AI
  EOS
```

因此真正的监督关系是：

```
位置0 → 我
  位置1 → 爱
  位置2 → AI
  位置3 → EOS
```

这就是标准的：

$$ \boxed{xt \rightarrow x{t+1}} $$

---

7. 为什么不能预测当前位置自己？

如果不 shift：

```
输入 token：我
  目标 token：我
```

模型最容易学成 Identity Mapping：

> “看见‘我’，就输出‘我’。”

这毫无意义。

语言模型真正需要学习的是：

```
根据当前位置及之前的信息
  → 预测下一个 token
```

也就是：

$$ P(x{t+1}\mid x{\le t}) $$

---

8. Tokenizer：文字如何进入神经网络？

神经网络不能直接处理：

```
Transformer 是一种神经网络
```

必须先转成整数。

MiniMind 的 tokenizer 是 BPE tokenizer，词表大小：

$$ V=6400 $$

源码配置中：

```
self.vocab_size = 6400
```

MiniMind 当前 tokenizer 的几个重要特殊 token：

```
ID 0 = <|endoftext|>    (同时作为 PAD / UNK)
  ID 1 = <|im_start|>     (BOS)
  ID 2 = <|im_end|>       (EOS)
```

也就是说：

```
bos_token_id = 1
  eos_token_id = 2
  pad_token_id = 0
```

---

9. MiniMind PretrainDataset 逐行分析

源码：

```
class PretrainDataset(Dataset):
      def __init__(self, data_path, tokenizer, max_length=512):
          super().__init__()
          self.tokenizer = tokenizer
          self.max_length = max_length
          self.samples = load_dataset(
              'json',
              data_files=data_path,
              split='train'
          )
```

数据格式来自 README：

```
{"text": "如何才能摆脱拖延症？治愈拖延症并不容易，但以下建议可能有所帮助。"}
  {"text": "清晨的阳光透过窗帘洒进房间，桌上的书页被风轻轻翻动。"}
  {"text": "Transformer 通过自注意力机制建模上下文关系，是现代大语言模型的重要基础结构。"}
```

因此每一行就是一个：

```
sample['text']
```

---

10. 文本 Tokenization

关键代码：

```
tokens = self.tokenizer(
      str(sample['text']),
      add_special_tokens=False,
      max_length=self.max_length - 2,
      truncation=True
  ).input_ids
```

这里有三个值得讲的地方。

10.1 `add_special_tokens=False`

告诉 tokenizer：

> *不要自动加 BOS / EOS。*

因为后面 MiniMind 自己手动加。

10.2 `max_length=self.max_length - 2`

为什么减 2？

因为还要留两个位置：

```
BOS
  EOS
```

如果最终长度最大为：

$$ L=340 $$

则正文最多：

$$ 340-2=338 $$

个 token。

10.3 `truncation=True`

正文太长时截断。

所以 MiniMind 当前 pretrain 并不是：

> *把超长文档连续切成很多 block 全部利用。*

而是：

> *每条 JSONL 文本最多取前 `max_length-2` 个 token。*

这是一个非常简洁、适合教学和低成本训练的数据方案。

---

11. 手动加入 BOS / EOS

源码：

```
tokens = (
      [self.tokenizer.bos_token_id]
      + tokens
      + [self.tokenizer.eos_token_id]
  )
```

例如：

> *原文：*
>
> *北京是中国的首都*
>
> *token：*
>
> *[100, 215, 378, 421]*
>
> *加入特殊 token：*
>
> *[1, 100, 215, 378, 421, 2]*

其中：

```
1 = BOS
  2 = EOS
```

BOS 告诉模型：

> *一个序列开始了。*

EOS 告诉模型：

> *这个序列应该结束了。*

模型因此不仅学会：

> *“接下来应该说什么”*

也学会：

> *“什么时候应该停止”。*

---

12. Padding

MiniMind 需要组成 batch。

一个 batch 中每个 Tensor 的长度必须一样。

假设：

$$ L_{\max}=10 $$

原 token：

```
[1, 100, 215, 378, 421, 2]
```

只有 6 个 token。

因此补 4 个 PAD：

```
[1, 100, 215, 378, 421, 2, 0, 0, 0, 0]
```

源码：

```
input_ids = tokens + [self.tokenizer.pad_token_id] * (
      self.max_length - len(tokens)
  )
```

---

13. 为什么 PAD 不能计算 loss？

如果 PAD 也参与 loss，模型会学：

```
EOS 后面应该疯狂预测 PAD
```

短文本越多，PAD 越多，这会严重污染训练目标。

因此：

```
labels = input_ids.clone()
  labels[input_ids == self.tokenizer.pad_token_id] = -100
```

例如：

```
input_ids:
  [1, 100, 215, 378, 421, 2, 0, 0, 0, 0]

  labels:
  [1, 100, 215, 378, 421, 2, -100, -100, -100, -100]
```

PyTorch 后面使用：

```
F.cross_entropy(..., ignore_index=-100)
```

因此 target 为 `-100` 的位置不计算 loss。

---

14. 完整的数据样例

假设：

```
文本：
  我爱AI
```

为了教学，假设 tokenizer 得到：

```
我  → 101
  爱  → 205
  AI → 399
```

最大长度：

$$ L=8 $$

那么 `PretrainDataset` *返回：*

```
input_ids =
  [1, 101, 205, 399, 2, 0, 0, 0]

  labels =
  [1, 101, 205, 399, 2, -100, -100, -100]
```

到了模型内部：

```
x = logits[..., :-1, :]
  y = labels[..., 1:]
```

对应：

```
x 有 7 个位置：

  x0
  x1
  x2
  x3
  x4
  x5
  x6

  y =
  [101, 205, 399, 2, -100, -100, -100]
```

实际计算 loss 的只有：

```
x0 → 101（我）
  x1 → 205（爱）
  x2 → 399（AI）
  x3 → 2（EOS）
```

后面三个：

```
-100
```

全部忽略。

---

15. Batch 后 Tensor 的形状

MiniMind 默认：

```
batch_size = 32
  max_seq_len = 340
  vocab_size = 6400
  hidden_size = 768
```

因此进入模型：

$$ \text{input_ids.shape} = [B,L] = [32,340] $$

Embedding 后：

$$ [32,340] \rightarrow [32,340,768] $$

最终 logits：

$$ [32,340,768] \rightarrow [32,340,6400] $$

shift 后：

$$ x.shape=[32,339,6400] $$

$$ y.shape=[32,339] $$

flatten：

$$ x: [32\times339,6400] = [10848,6400] $$

$$ y: [10848] $$

所以 PyTorch 的 CrossEntropy 实际上看到的是：

> ***10848 道 6400 分类题。***

当然其中 PAD 对应的位置因为 label = -100 被忽略。

---

16. 模型输出的 logits 是什么？

对每个位置，模型输出一个 6400 维向量：

$$ z=(z1,z2,\ldots,z_{6400}) $$

例如：

```
[2.1, -0.3, 4.7, ..., 1.2]
```

它们不是概率。

叫：

> logits。

通过 softmax：

$$ p_i= \frac{e^{z_i}} {\sum{j=1}^{V}e^{zj}} $$

得到：

$$ \sumi pi=1 $$

例如模型认为下一个 token：

```
“是”        0.60
  “的”        0.10
  “中国”      0.08
  ……
```

如果正确答案是“是”，那么该位置 loss：

$$ L=-\log(0.60) $$

概率越高：

$$ -\log p $$

越小。

---

17. Cross Entropy 到底是什么？

假设词表只有四个词：

```
[我, 爱, AI, 猫]
```

正确答案：

```
AI
```

one-hot：

$$ y=[0,0,1,0] $$

模型预测：

$$ p=[0.1,0.1,0.7,0.1] $$

交叉熵：

$$ H(y,p) = -\sum yi\log p_i $$

因为只有正确类别的 (y_i=1)，所以：

$$ L=-\log 0.7 $$

也就是说：

$$ \boxed{ L=-\log P(\text{正确 token}) } $$

这是理解语言模型 loss 最简单的方式。

---

18. 一个完整的数值例子

序列：

```
<BOS> 我 爱 AI <EOS>
```

假设模型四个位置对正确答案给出的概率分别为：

```
P(我 | BOS)           = 0.60
  P(爱 | BOS,我)        = 0.70
  P(AI | BOS,我,爱)     = 0.80
  P(EOS | BOS,我,爱,AI)= 0.50
```

平均 loss：

$$ L = -\frac14 (\log0.6+ \log0.7+ \log0.8+ \log0.5) $$

得到约：

$$ \boxed{L\approx0.446} $$

Perplexity：

$$ PPL=e^L $$

所以：

$$ PPL\approx1.562 $$

直观理解：

> 模型在每一步平均像是在约 1.56 个同等可能的候选中做选择。

注意真实 LLM 词表有 6400 个 token，情况当然复杂得多。

---

19. 为什么 CrossEntropy 代码里没有 Softmax？

MiniMind：

```
loss = F.cross_entropy(...)
```

学生常问：

> 前面没写 softmax，怎么就是概率？

因为：

```
F.cross_entropy(logits, target)
```

数学上等价于：

```
log_softmax
  +
  negative log likelihood
```

即：

$$ \text{CrossEntropy}(z,y) = -\log \frac{e^{z_y}} {\sumj e^{zj}} $$

PyTorch 直接从 logits 计算，数值更稳定。

所以：

> **训练时不要先自己 softmax 再传给 `F.cross_entropy`。**

---

20. 为什么一个位置不能偷看未来？

假设训练：

> 我 爱 人工智能

当模型在“我”这个位置预测“爱”时，如果 Attention 可以看到右侧：

> 爱

那答案已经泄露了。

模型会作弊。

因此 Decoder-only Transformer 必须使用 causal mask。

---

21. Causal Mask

长度为 4 时，注意力可见关系：

```
       K0  K1  K2  K3

Q0      ✓   ×   ×   ×
Q1      ✓   ✓   ×   ×
Q2      ✓   ✓   ✓   ×
Q3      ✓   ✓   ✓   ✓
```

数学上 mask：

$$ M= \begin{bmatrix} 0&-\infty&-\infty&-\infty\ 0&0&-\infty&-\infty\ 0&0&0&-\infty\ 0&0&0&0 \end{bmatrix} $$

Attention：

$$ \boxed{ \mathrm{Attention}(Q,K,V) = \mathrm{softmax} \left(\frac{QK^\top}{\sqrt{d_k}} + M \right)V } $$

因为：

$$ e^{-\infty}=0 $$

右上角未来位置 softmax 后概率变成 0。

---

22. MiniMind 如何实现 causal attention？

MiniMind：

```
self.is_causal = True
```

如果支持 PyTorch Flash / SDPA：

```
F.scaled_dot_product_attention(
      xq,
      xk,
      xv,
      ...,
      is_causal=self.is_causal
  )
```

这里：

```
is_causal=True
```

由 PyTorch 完成 causal mask。

如果不用 Flash Attention，则自己构造：

```
scores = (xq @ xk.transpose(-2, -1)) / math.sqrt(self.head_dim)

  scores[:, :, :, -seq_len:] += torch.full(
      (seq_len, seq_len),
      float("-inf"),
      device=scores.device
  ).triu(1)
```

核心就是：

```
triu(1)
```

得到严格上三角矩阵。

也就是把未来位置变成：

```
-inf
```

---

23. MiniMind 的整体模型结构

默认配置：

```
hidden_size = 768
  num_hidden_layers = 8
  vocab_size = 6400

  num_attention_heads = 8
  num_key_value_heads = 4

  head_dim = 768 / 8 = 96
```

中间层：

```
intermediate_size =
  ceil(hidden_size * pi / 64) * 64
```

代入：

$$ 768\times\pi\approx2412.7 $$

向上对齐到 64：

$$ \boxed{2432} $$

所以默认 Dense MiniMind：

```
Vocabulary: 6400
  Hidden:     768
  Layers:     8
  Q heads:    8
  KV heads:   4
  Head dim:   96
  FFN dim:    2432
```

---

24. 为什么参数量约 64M？

Embedding

$$ 6400\times768 = 4,915,200 $$

因为 MiniMind：

```
if self.config.tie_word_embeddings:
      self.model.embed_tokens.weight = self.lm_head.weight
```

输入 embedding 和输出 LM Head 共用同一份参数。

因此不是两份 (6400\times768)，而是一份。

每层 Attention 参数

Q：

$$ 768\times(8\times96) = 768\times768 = 589,824 $$

K：

$$ 768\times(4\times96) = 768\times384 = 294,912 $$

V：

$$ 294,912 $$

O：

$$ 768\times768 = 589,824 $$

总计：

$$ 1,769,472 $$

每层 FFN

MiniMind 使用类似 SwiGLU 的三矩阵结构：

```
gate_proj
  up_proj
  down_proj
```

每个：

$$ 768\times2432 = 1,867,776 $$

三份：

$$ 5,603,328 $$

---

8 层 Transformer

再加 RMSNorm 等小量参数后，每层约：

$$ 7.376M $$

8 层：

$$ \approx59.01M $$

加 embedding 和最终 norm：

$$ \boxed{ \approx63.93M } $$

因此 README 中称默认模型约：

$$ \boxed{64M} $$

是完全对应源码的。

---

25. Embedding：Token ID 如何变成向量？

输入：

```
101
```

只是一个整数。

神经网络需要向量。

MiniMind：

```
self.embed_tokens = nn.Embedding(
      config.vocab_size,
      config.hidden_size
  )
```

Embedding Matrix：

$$ E\in\mathbb{R}^{6400\times768} $$

一个 token id (x_t) 查表：

$$ ht=E[xt] $$

得到：

$$ h_t\in\mathbb{R}^{768} $$

所以：

$$ [B,L] \rightarrow [B,L,768] $$

源码：

```
hidden_states = self.dropout(
      self.embed_tokens(input_ids)
  )
```

---

26. 为什么需要位置编码？

Self-Attention 本身并不知道：

```
我 爱 你
```

和：

```
你 爱 我
```

谁在前谁在后。

所以必须加入位置信息。

MiniMind 使用：

> ***RoPE：Rotary Position Embedding***

不是传统 Transformer 那种：

$$ x + PE $$

而是对 Query / Key 做旋转。

---

27. RoPE 的直观理解

对二维向量：

$$ \begin{bmatrix} x1\x2 \end{bmatrix} $$

旋转角度 (\theta)：

$$ R(\theta)= \begin{bmatrix} \cos\theta&-\sin\theta\ \sin\theta&\cos\theta \end{bmatrix} $$

RoPE 给不同位置不同旋转角度。

位置 (m)：

$$ qm'=R(m\theta)qm $$

位置 (n)：

$$ kn'=R(n\theta)kn $$

二者点积天然包含：

$$ m-n $$

即相对位置信息。

---

28. MiniMind 的 RoPE 代码

初始化时预计算：

```
freqs_cos, freqs_sin = precompute_freqs_cis(
      dim=config.head_dim,
      end=config.max_position_embeddings,
      rope_base=config.rope_theta,
      rope_scaling=config.rope_scaling
  )
```

默认：

```
head_dim = 96
  max_position_embeddings = 32768
  rope_theta = 1e6
```

forward：

```
xq, xk = apply_rotary_pos_emb(
      xq,
      xk,
      cos,
      sin
  )
```

值得强调：

> *RoPE 只作用于 Q 和 K，不直接作用于 V。*

因为位置信息需要改变：

> *“谁应该关注谁”*

而 Attention 权重来自 Q 和 K。

---

29. Q、K、V 是怎么来的？

输入：

$$ X\in\mathbb{R}^{B\times L\times768} $$

线性投影：

$$ Q=XW_Q $$

$$ K=XW_K $$

$$ V=XW_V $$

MiniMind：

```
xq = self.q_proj(x)
  xk = self.k_proj(x)
  xv = self.v_proj(x)
```

---

30. Multi-Head Attention

MiniMind Q 有 8 个头：

```
num_attention_heads = 8
```

每头：

```
head_dim = 96
```

所以：

$$ 8\times96=768 $$

Q reshape：

$$ [B,L,768] \rightarrow [B,L,8,96] $$

源码：

```
xq = xq.view(
      bsz,
      seq_len,
      self.n_local_heads,
      self.head_dim
  )
```

---

31. MiniMind 不是普通 MHA，而是 GQA

MiniMind 默认：

```
Q heads  = 8
  KV heads = 4
```

这叫：

> ***Grouped Query Attention，GQA***

即多个 Q head 共用较少的 K/V head。

MiniMind：

```
self.n_rep = (
      self.n_local_heads //
      self.n_local_kv_heads
  )
```

因此：

$$ n_{rep}=8/4=2 $$

每个 KV head 被两个 Query head 使用。

---

32. GQA 为什么有用？

普通 Multi-Head Attention：

```
8 Q
  8 K
  8 V
```

GQA：

```
8 Q
  4 K
  4 V
```

优势：

- *K/V 参数更少*
- *推理时 KV Cache 更小*
- *生成速度更友好*
- *通常效果损失较小*

所以现代 LLM 很常见。
MiniMind 中：

```
repeat_kv(xk, self.n_rep)
  repeat_kv(xv, self.n_rep)
```

把 4 个 KV 逻辑复制为供 8 个 Q head 使用。

注意：

> 这不是增加新的参数，仅仅是计算上的展开。

---

33. QK Norm

MiniMind 在 Q/K 上还使用 RMSNorm：

```
self.q_norm = RMSNorm(self.head_dim)
  self.k_norm = RMSNorm(self.head_dim)
```

forward：

```
xq = self.q_norm(xq)
  xk = self.k_norm(xk)
```

这是现代 LLM 常见的稳定训练设计之一。

---

34. RMSNorm

传统 LayerNorm：

$$ \frac{x-\mu}{\sqrt{\sigma^2+\epsilon}} $$

RMSNorm 不减均值，只按照均方根归一化：

$$ \mathrm{RMS}(x) = \sqrt{ \frac1d \sum_{i=1}^d xi^2 + \epsilon } $$

$$ \boxed{ \mathrm{RMSNorm}(x) = g \odot \frac{x}{\mathrm{RMS}(x)} } $$

MiniMind：

```
return x * torch.rsqrt(
      x.pow(2).mean(-1, keepdim=True)
      + self.eps
  )
```

然后：

```
self.weight * normalized_x
```

---

35. Attention 公式

每个 head：

$$ S= \frac{QK^\top} {\sqrt{d_k}} $$

加 causal mask：

$$ S'=S+M $$

softmax：

$$ A=\mathrm{softmax}(S') $$

输出：

$$ O=AV $$

最终多头拼接后：

$$ \mathrm{Concat}(O1,\ldots,Oh)W_O $$

MiniMind 对应：

```
scores = (
      xq @ xk.transpose(-2, -1)
  ) / math.sqrt(self.head_dim)
```

```
output =
  self.attn_dropout(
      F.softmax(scores.float(), dim=-1).type_as(xq)
  ) @ xv
```

---

36. 为什么除以 (\sqrt{d_k})？

如果 Q 和 K 每个维度方差大致为 1：

$$ q\cdot k = \sum{i=1}^{dk}qi ki $$

其方差会随：

$$ d_k $$

增大。

(d_k) 大时 logits 绝对值会很大。
softmax 容易变得过尖：

```
[0.999999, 0.000001, ...]
```

梯度变差。

因此缩放：

$$ \frac{QK^\top}{\sqrt{d_k}} $$

使数值尺度更稳定。

MiniMind：

```
d_k = 96
```

所以除以：

$$ \sqrt{96} $$

---

37. Transformer Block 的结构

MiniMind 每层：

```
residual = hidden_states

  hidden_states, ... = self.self_attn(
      self.input_layernorm(hidden_states),
      ...
  )

  hidden_states += residual

  hidden_states = hidden_states + self.mlp(
      self.post_attention_layernorm(hidden_states)
  )
```

可以写为：

$$ H'_l = H_l + \mathrm{Attention}\left(\mathrm{RMSNorm}(H_l) \right) $$

然后：

$$ \boxed{ H_{l+1} = H'_l + \mathrm{FFN}\left(\mathrm{RMSNorm}(H'_l) \right) } $$

这属于：

> ***Pre-Norm Transformer***

Normalization 在子层之前。

---

38. 为什么需要 Residual Connection？

Attention 层不是直接替换旧表示，而是：

$$ x+\mathrm{Attention}(x) $$

FFN：

$$ x+\mathrm{FFN}(x) $$

Residual 的意义包括：

- *保留原始信息*
- *改善深层网络梯度传播*
- *让每层更像是在原表示上学习“增量修正”*

可以给学生一个直观说法：

> *每层 Transformer 不是把上一层推倒重建，而是在上一层知识表示上“补充一点新的理解”。*

---

39. MiniMind 的 FFN 不是普通两层 MLP

源码：

```
return self.down_proj(
      self.act_fn(self.gate_proj(x))
      * self.up_proj(x)
  )
```

默认：

```
hidden_act = 'silu'
```

所以：

$$ g=\mathrm{SiLU}(XW_g) $$

$$ u=XW_u $$

逐元素乘：

$$ z=g\odot u $$

最后：

$$ \boxed{ FFN(X)= (\mathrm{SiLU}(XW_g) \odot XW_u) W_d } $$

这是 SwiGLU 类结构。

---

40. SiLU

$$ \mathrm{SiLU}(x) = x\sigma(x) $$

其中：

$$ \sigma(x) = \frac1{1+e^{-x}} $$

和 ReLU 相比，SiLU 更平滑。

现代 LLM 中经常与 GLU 类门控结合。

---

41. 8 层 Transformer 后发生什么？

MiniMind：

```
for layer in self.layers:
      hidden_states, present = layer(...)
```

共：

```
8 layers
```

最后：

```
hidden_states = self.norm(hidden_states)
```

所以：

$$ [B,L,768] \rightarrow [B,L,768] $$

仍然每个 token 有一个 768 维 hidden state。

---

42. LM Head

语言模型最终必须回答：

> *下一个 token 是词表 6400 个 token 中的哪一个？*

所以需要：

```
self.lm_head = nn.Linear(
      hidden_size,
      vocab_size,
      bias=False
  )
```

即：

$$ 768 \rightarrow 6400 $$

因此：

$$ H\in \mathbb{R}^{B\times L\times768} $$

经过 LM Head：

$$ Z=HW^T $$

得到：

$$ \boxed{ Z\in \mathbb{R}^{B\times L\times6400} } $$

每个位置都是一次 6400 分类。

---

43. Weight Tying

MiniMind：

```
if self.config.tie_word_embeddings:
      self.model.embed_tokens.weight = self.lm_head.weight
```

意味着：

> *输入 token 用的词向量矩阵*
> *和*
> *输出 token 分类用的矩阵*

共享参数。

输入：

$$ E\in\mathbb R^{V\times H} $$

输出：

$$ z=hE^T $$

这叫：

> *Weight Tying / Tied Embedding。*

优点：

- *减少参数量*
- *输入输出语义空间共享*
- *小模型尤其划算*

MiniMind 因此节约约：
$$ 6400\times768 = 4.92M $$

参数。

---

44. 一次 forward 的完整形状变化

默认 batch：

```
B = 32
  L = 340
  H = 768
  V = 6400
  Q heads = 8
  KV heads = 4
  head dim = 96
```

输入

$$ input_ids: [32,340] $$

Embedding

$$ [32,340] \rightarrow [32,340,768] $$

Q

$$ [32,340,768] \rightarrow [32,340,768] \rightarrow [32,340,8,96] $$

K/V

$$ [32,340,768] \rightarrow [32,340,384] \rightarrow [32,340,4,96] $$

repeat K/V

$$ [32,340,4,96] \rightarrow [32,340,8,96] $$

transpose

$$ [32,8,340,96] $$

Attention score

$$ QK^\top: [32,8,340,340] $$

Attention 输出

$$ [32,8,340,96] $$

拼接 heads

$$ [32,340,768] $$

8 层后

$$ [32,340,768] $$

LM Head

$$ [32,340,6400] $$

Shift

$$ logits: [32,339,6400] $$

$$ labels: [32,339] $$

---

45. 为什么训练可以同时预测所有 token，而推理不能？

这是一个非常重要的课堂问题。

训练

我们已经知道完整文本：

```
我 爱 人工智能
```

借助 causal mask，可以一次输入整个序列：

```
我
  我 爱
  我 爱 人工智能
```

对应的所有位置可以矩阵并行。

推理

假设只有：

```
我
```

后面的“爱”还不存在。

必须先生成：

```
爱
```

然后才能生成下一个：

> 人工智能

所以生成天然是：

$$ x_1 \rightarrow x_2 \rightarrow x_3 \rightarrow \cdots $$

自回归过程。

---

46. Pretrain 为什么能学到“知识”？

例如训练语料里反复出现：

> 法国的首都是巴黎
> 巴黎是法国最大的城市之一
> 埃菲尔铁塔位于巴黎
> ……

模型在大量 next-token prediction 中不断降低：

$$ -\log P(\text{巴黎} \mid \text{法国的首都是}) $$

为了预测准确，它必须从参数中捕捉：

> 法国 ↔ 巴黎
> 巴黎 ↔ 埃菲尔铁塔

等统计关联。

因此语言模型知识不是传统数据库：

> key → value

而是分布式编码在大量神经网络权重中的。

---

47. 为什么 next-token prediction 可以产生“涌现能力”？

因为要准确预测下一个 token，很多时候模型不能只记局部词频。

例如：

```
小明有3个苹果，又买了2个，现在有____
```

如果训练数据丰富，模型为了预测：

```
5
```

可能需要内部表示：

- 数字
- 加法
- 语义关系
- 文本结构

再例如代码：

```
for i in range(10):
    print(
```

要预测后面的代码，需要学习：

- Python 语法
- 括号
- API 模式
- 缩进
- 上下文变量

所以一个非常简单的统一训练目标：
$$ \boxed{ \text{Predict Next Token} } $$

可以迫使网络学习非常复杂的内部表示。

---

48. MiniMind 的训练主程序

`train_pretrain.py` 可以分为 9 个阶段：

```
1. 初始化环境和随机种子
2. 创建模型配置 / 检查断点
3. 配置混合精度
4. 配置实验日志
5. 创建模型 / Dataset / Optimizer
6. 恢复 checkpoint
7. torch.compile / DDP
```

```
8. epoch 训练循环
9. 清理分布式进程
```

---

49. 命令行参数逐项解释

默认：

```
--epochs 2
  --batch_size 32
  --learning_rate 5e-4
  --dtype bfloat16
  --accumulation_steps 8
  --grad_clip 1.0
  --hidden_size 768
  --num_hidden_layers 8
  --max_seq_len 340
  --use_moe 0
```

---

50. Epoch

```
--epochs 2
```

一个 epoch：

> 整个训练数据集大致被访问一遍。

2 epochs：

> 数据集训练两轮。

注意大模型预训练通常更习惯用：

```
tokens
  steps
  FLOPs
```

衡量训练规模，而不只用 epoch。

因为大规模语料可能一轮就已经非常巨大。

---

51. Batch Size

```
--batch_size 32
```

每个 micro-batch：

$$ 32 $$

条序列。

最大 token 数：

$$ 32\times340 = 10880 $$

注意其中可能包含 PAD。

---

52. 梯度累积

MiniMind：

```
--accumulation_steps 8
```

每 8 个 batch 才：

```
optimizer.step()
```

因此单卡时名义有效 batch size：

$$ 32\times8 = \boxed{256} $$

如果使用：

$$ N $$

张 GPU DDP，则全局 batch size 约：

$$ \boxed{ 32\times8\times N } $$

例如 4 GPU：

$$ 32\times8\times4=1024 $$

条序列 / optimizer update。

53. 梯度累积为什么先除以 accumulation_steps？

源码：

```
loss = loss / args.accumulation_steps
```

然后：

```
loss.backward()
```

假设 8 个 micro-batch：

$$ gi=\nabla Li $$

如果不除 8：

$$ g= \sum{i=1}^8 gi $$

梯度规模会放大约 8 倍。

除以 8：

$$ g= \frac18 \sum{i=1}^8 gi $$

相当于大 batch 平均梯度。

---

54. 训练循环核心

MiniMind：

```
for step, (input_ids, labels) in enumerate(loader):
```

每步：

```
input_ids = input_ids.to(args.device)
  labels = labels.to(args.device)
```

GPU 上计算。

---

55. 学习率调度

MiniMind：

```
lr = get_lr(
      epoch * iters + step,
      args.epochs * iters,
      args.learning_rate
  )
```

`get_lr`：

```
return lr * (
      0.1
      + 0.45 * (
          1 + cos(pi * current_step / total_steps)
      )
  )
```

数学形式：

$$ \boxed{ \eta_t = \eta_0 \left[0.1+ 0.45 \left(1+\cos\frac{\pi t}{T} \right) \right] } $$

---

56. 这个 cosine schedule 从多少降到多少？

当：

$$ t=0 $$

时：

$$ \cos0=1 $$

所以：

$$ 0.1+0.45(2)=1 $$

$$ \eta_0=5\times10^{-4} $$

训练末尾：

$$ t=T $$

$$ \cos\pi=-1 $$

所以：

$$ 0.1+0=0.1 $$

最终学习率：

$$ 0.1\eta_0 = 5\times10^{-5} $$

所以 MiniMind 当前实现是：

> **无 warmup 的 cosine decay，从初始 LR 平滑衰减到初始值的 10%。**

这是源码事实。

课堂上可以顺便指出：

> 大型正式预训练通常经常还会加入 warmup；MiniMind 为了保持训练代码极简，这里没有实现 warmup。

---

57. Forward + Loss

核心：

```
with autocast_ctx:
      res = model(input_ids, labels=labels)
      loss = res.loss + res.aux_loss
      loss = loss / args.accumulation_steps
```

Dense 模型时：

```
aux_loss = 0
```

所以：

$$ loss = LM\ Loss $$

MoE 时：

$$ loss = LM\ Loss + Router\ Auxiliary\ Loss $$

---

58. `res.loss` 在哪里计算？

不是 `train_pretrain.py` 计算。

而是在：

```
model/model_minimind.py
  MiniMindForCausalLM.forward()
```

内部：

```
if labels is not None:
      x = logits[..., :-1, :]
      y = labels[..., 1:]
      loss = F.cross_entropy(...)
```

这是理解项目结构的重要一点：

```
Trainer
    负责优化流程


  Model
    负责 forward 和语言模型 loss


  Dataset
    负责把原始数据做成 input_ids / labels
```

---

59. 反向传播

MiniMind：

```
scaler.scale(loss).backward()
```

从 CrossEntropy 开始，梯度一路反传：

```
CrossEntropy
        ↓
  LM Head
        ↓
  第8层 Transformer
        ↓
  ……
        ↓
  第1层 Transformer
        ↓
  Embedding
```

用链式法则：

$$ \frac{\partial L}{\partial W} = \frac{\partial L}{\partial z} \frac{\partial z}{\partial h} \frac{\partial h}{\partial W} $$

最终每个可训练参数都获得梯度。

---

60. Softmax + Cross Entropy 的梯度为什么漂亮？

设 logits：

$$ z_i $$

softmax：

$$ p_i $$

真实标签 one-hot：

$$ y_i $$

Cross Entropy 对 logits 的梯度：

$$ \boxed{ \frac{\partial L}{\partial z_i} = pi-yi } $$

如果正确 token 是第 3 类：

```
y = [0,0,1,0]
```

模型：

```
p = [0.1,0.1,0.7,0.1]
```

梯度：

```
[+0.1, +0.1, -0.3, +0.1]
```

含义：

- 错误 token 的 logits 往下压
- 正确 token 的 logits 往上推

模型就这样一步一步变得更准确。

---

61. AdamW

MiniMind：

```
optimizer = optim.AdamW(
      model.parameters(),
      lr=args.learning_rate
  )
```

Adam 的核心是维护：

一阶动量：

$$ m_t = \beta1m{t-1} + (1-\beta1)gt $$

二阶动量：

$$ v_t = \beta2v{t-1} + (1-\beta2)gt^2 $$

参数更新大致：

$$ \theta_{t+1} = \theta_t - \eta \frac{\hat m_t} {\sqrt{\hat v_t}+\epsilon} $$

AdamW 将 weight decay 与梯度更新解耦。

MiniMind 没有显式填写 betas / weight_decay，所以使用 PyTorch `AdamW` 默认值。

---

62. 梯度裁剪

MiniMind：

```
torch.nn.utils.clip_grad_norm_(
      model.parameters(),
      args.grad_clip
  )
```

默认：

```
grad_clip = 1.0
```

如果整体梯度范数：

$$ |g|_2>1 $$

就按比例缩小。

例如：

$$ |g|=5 $$

大约乘：

$$ \frac15 $$

使 norm 变为 1。

目的：

> *防止偶发的大梯度让参数更新爆炸。*

---

63. 为什么先 `scaler.unscale_` 再梯度裁剪？

float16 混合精度训练时，GradScaler 会把 loss 放大。

如果直接 clip：

> *clip 的其实是被放大的梯度。*

所以 MiniMind：

```
scaler.unscale_(optimizer)
  torch.nn.utils.clip_grad_norm_(...)
```

先恢复真实梯度尺度，再 clip。

这是正确顺序。

64. Mixed Precision

MiniMind：

```
dtype = (
      torch.bfloat16
      if args.dtype == "bfloat16"
      else torch.float16
  )
```

GPU 上：

```
torch.cuda.amp.autocast(dtype=dtype)
```

混合精度目的：

- *降低显存*
- *提高 Tensor Core 吞吐*
- *加速训练*

65. FP16 与 BF16

FP16：

```
1 sign
  5 exponent
  10 mantissa
```

BF16：

```
1 sign
  8 exponent
  7 mantissa
```

BF16 和 FP32 都有 8 位 exponent，因此数值范围很大，更不容易 overflow。

所以现代大模型训练如果硬件支持，通常更偏向 BF16。

MiniMind 默认：

```
bfloat16
```

---

66. 为什么 BF16 时不启用 GradScaler？

MiniMind：

```
scaler = torch.cuda.amp.GradScaler(
      enabled=(args.dtype == 'float16')
  )
```

也就是说：

```
FP16 → GradScaler 开启
  BF16 → GradScaler 关闭
```

原因是 FP16 的指数范围更小，容易下溢。

BF16 范围更接近 FP32，一般不需要 loss scaling。

---

67. Optimizer Step

只有：

```
if step % accumulation_steps == 0:
```

才执行：

```
scaler.step(optimizer)
  scaler.update()
  optimizer.zero_grad(set_to_none=True)
```

所以流程：

```
micro batch 1 → backward
  micro batch 2 → backward
  ...
  micro batch 8 → backward
                    ↓
                clip grad
                    ↓
             optimizer.step
                    ↓
               zero_grad
```

---

68. `zero_grad(set_to_none=True)` 为什么不是清零？

普通：

```
grad.zero_()
```

会保留 gradient tensor，只把值改为 0。

`set_to_none=True`：

```
param.grad = None
```

通常更省内存 / 更高效。

下一次 backward 再创建 gradient。

---

69. 一个 optimizer step 到底用了多少 token？

默认单 GPU：

```
batch_size = 32
  seq_len = 340
  accumulation = 8
```

最大输入 token：

$$ 32\times340\times8 = \boxed{87040} $$

token / optimizer update。

严格说：

- *PAD 不产生有效语言模型 loss*
- *shift 后每条序列最多是 339 个预测 target*

最大有效预测位置接近：
$$ 32\times339\times8 = 86784 $$

如果全都没有 padding。

---

70. DDP 多 GPU

MiniMind：

```
local_rank = init_distributed_mode()
```

如果环境变量有 RANK：

```
dist.init_process_group(
      backend="nccl"
  )
```

然后：

```
model = DistributedDataParallel(
      model,
      device_ids=[local_rank]
  )
```

DDP 基本思想：

```
GPU 0：一部分 batch
  GPU 1：一部分 batch
  GPU 2：一部分 batch
  GPU 3：一部分 batch
          ↓
  各自 forward/backward
          ↓
  梯度 All-Reduce
          ↓
  每张卡得到相同平均梯度
          ↓
  各自 optimizer.step()
          ↓
  参数继续保持一致
```

---

71. DistributedSampler

MiniMind：

```
train_sampler = DistributedSampler(train_ds)
```

作用：

> *不同 GPU 不要重复吃同一批数据。*

例如数据：

```
0 1 2 3 4 5 6 7
```

两张 GPU 可以大致变为：

```
GPU0 → 0 2 4 6
  GPU1 → 1 3 5 7
```

实际 DistributedSampler 还会 shuffle。

---

72. 为什么每个 epoch 要 `set_epoch(epoch)` ？

MiniMind：

```
train_sampler.set_epoch(epoch)
```

DistributedSampler 的随机 shuffle 与 epoch 相关。

如果不调用：

> *每一轮可能使用相同的 shuffle 顺序。*

因此 DDP 训练中这是标准做法。

---

73. 单 GPU 时怎么 Shuffle？

源码：

```
setup_seed(args.seed + epoch)
  indices = torch.randperm(len(train_ds)).tolist()
```

也就是说：

> *每个 epoch 先得到一个随机样本顺序。*

然后 `SkipBatchSampler` *按这个顺序组成 batch。*

---

74. 断点续训

MiniMind 有两类文件：

> *普通模型权重：*
>
> `pretrain_768.pth`
>
> *恢复训练* `checkpoint`*：*
>
> `pretrain_768_resume.pth`

恢复 checkpoint 包括：

```
{
      'model': ...,
      'optimizer': ...,
      'epoch': ...,
      'step': ...,
      'world_size': ...,
      'wandb_id': ...
  }
```

训练恢复时不仅恢复模型：

```
model.load_state_dict(...)
```

也恢复：

```
optimizer.load_state_dict(...)
  scaler.load_state_dict(...)
```

为什么必须恢复 optimizer？

因为 AdamW 不是只有模型参数。

它还有：

```
一阶动量 m
  二阶动量 v
```

如果只恢复模型权重、不恢复 optimizer：

> *虽然看似“继续训练”，实际上 Adam 的历史状态丢了。*

---

75. SkipBatchSampler

断点中已经完成前 1000 个 batch。

重新打开数据集后不能从头再训练。

MiniMind：

```
SkipBatchSampler(
      sampler,
      batch_size,
      skip
  )
```

会跳过已经训练过的 batch。

因此可以从：

```
step 1001
```

继续。

---

76. 保存模型前为什么处理 DDP / compile？

源码：

```
raw_model = (
      model.module
      if isinstance(model, DistributedDataParallel)
      else model
  )
```

DDP 会在真实模型外套一层包装。

要保存真正模型：

```
model.module
```

如果模型被 `torch.compile` *包装，还进一步：*

```
getattr(raw_model, '_orig_mod', raw_model)
```

取回原始 module。

---

77. 为什么保存成 half + CPU？

MiniMind：

```
{k: v.half().cpu() for k, v in state_dict.items()}
```

含义：

```
GPU Tensor
    ↓
  FP16
    ↓
  CPU
    ↓
  torch.save
```

优点：

- *权重文件更小*
- *保存时不把 CUDA tensor 状态直接写进去*
- *后续更方便加载*

例如 64M 参数：
FP32 大约：

$$ 64M\times4 Byte \approx256MB $$

FP16：

$$ 64M\times2 Byte \approx128MB $$

忽略序列化开销时约减半。

---

78. `model.train()` / `model.eval()`

保存前：

```
model.eval()
```

保存后：

```
model.train()
```

对于当前默认：

```
dropout = 0
```

差异不大。

但这是良好的模型状态管理习惯。

如果 dropout > 0：

- *train：启用 Dropout*
- *eval：关闭 Dropout*

---

79. MoE 模式与 Pretrain Loss

默认：

```
--use_moe 0
```

不使用 MoE。

如果：

```
--use_moe 1
```

每层 FFN 换成：

```
MOEFeedForward
```

router：

```
scores = F.softmax(
      self.gate(x_flat),
      dim=-1
  )
```

选择 Top-K expert：

```
topk_weight, topk_idx = torch.topk(...)
```

默认：

```
num_experts = 4
  num_experts_per_tok = 1
```

因此每个 token 只路由到一个 expert。

---

80. MoE Auxiliary Loss

MoE 如果不加约束，router 可能：

> *所有 token 都选 expert 0*

其他 expert 闲置。

MiniMind 添加负载平衡 auxiliary loss：

```
load = F.one_hot(
      topk_idx,
      num_experts
  ).float().mean(0)

  aux_loss = (
      load * scores.mean(0)
  ).sum() * num_experts * coef
```

训练：

$$ \boxed{ L_{total} = L_{LM} + L_{aux} } $$

源码：

```
loss = res.loss + res.aux_loss
```

默认 Dense：

$$ L_{aux}=0 $$

---

81. Pretrain 和 SFT 的核心区别：Loss Mask

这是课堂上非常值得并排讲的一点。

Pretrain

MiniMind：

```
labels = input_ids.clone()
  labels[PAD] = -100
```

除了 padding：

> ***几乎每个 token 都需要预测。***

例如：

```
Transformer 是一种神经网络
```

所有正文 token 都训练。

SFT

SFTDataset 的 `generate_labels()` *会先：*

```
labels = [-100] * len(input_ids)
```

只把：

```
assistant 回答部分
```

改回真实 token id。

也就是说 SFT 通常不要求模型学习：

```
user 问题本身应该如何生成
```

主要要求模型学习：

```
给定 system + user
  → assistant 应该回答什么
```

---

82. 一个最直观的 Pretrain vs SFT 对比

Pretrain

文本：

> 北京是中国的首都

loss：

> BOS → 北京
> 北京 → 是
> 是 → 中国
> 中国 → 的
> 的 → 首都
> 首都 → EOS

几乎全算。

SFT

> user: 中国的首都是哪里？
> assistant: 北京。

loss 常见做法：

> system/user 部分 → mask
> assistant 的“北京。” → 计算 loss

所以：

> Pretrain 学“语言和知识分布”。

> SFT 学“按照指令回答”。

---

83. 为什么 Pretrain 后模型还不一定会聊天？

因为预训练目标只是：

> 预测互联网文本接下来是什么。

它可能看到：

```
问：1+1等于多少？
  答：2
```

也可能看到：

```
有人问“1+1等于多少？”
  这个问题很简单……
```

也可能看到文章、小说、网页、代码。

所以 Base Model 学的是：

$$ P(\text{text}) $$

而不是明确学：

$$ P(\text{helpful answer}\mid \text{user instruction}) $$

SFT 才把行为分布拉向：

```
User → Assistant
```

---

84. MiniMind Pretrain 的完整调用图

```
train_pretrain.py
  |
  ├── MiniMindConfig(...)
  |
  ├── init_model(...)
  |   |
  |   ├── AutoTokenizer.from_pretrained("../model")
  |   └── MiniMindForCausalLM(config)
  |
  ├── PretrainDataset(...)
  |   |
  |   ├── load_dataset(json)
  |   ├── tokenizer(text)
  |   ├── + BOS
  |   ├── + EOS
  |   ├── + PAD
  |   └── PAD label → -100
  |
  ├── DataLoader
  |
  └── train_epoch(...)
      |
      ├── input_ids.to(GPU)
      ├── labels.to(GPU)
      ├── get_lr(...)
      |
      ├── model(input_ids, labels)
      |   |
      |   ├── Embedding
      |   ├── Transformer Block × 8
      |   |   ├── RMSNorm
      |   |   ├── GQA + RoPE + causal attention
      |   |   ├── residual
      |   |   ├── RMSNorm
      |   |   ├── SwiGLU FFN
      |   |   └── residual
      |   ├── final RMSNorm
      |   ├── LM Head
      |   ├── logits[:, :-1]
      |   ├── labels[:, 1:]
```

```
    │   └── CrossEntropy
    │
    ├── loss / accumulation_steps
    ├── backward()
    │
    ├── 每 accumulation_steps：
    │   ├── unscale
    │   ├── grad clip
    │   ├── AdamW.step
    │   └── zero_grad
    │
    ├── log
    └── checkpoint
```

---

85. 用伪代码把 MiniMind Pretrain 缩到 20 行

```
tokenizer = load_tokenizer()
  model = MiniMindForCausalLM(config)
  optimizer = AdamW(model.parameters())

  for text in corpus:

      ids = tokenizer(text)

      ids = [BOS] + ids + [EOS]
      ids = pad(ids)

      labels = ids.copy()
      labels[PAD] = -100

      logits = model(ids)

      pred = logits[:-1]
      target = labels[1:]

      loss = cross_entropy(
          pred,
          target,
          ignore_index=-100
      )

      loss.backward()
      optimizer.step()
      optimizer.zero_grad()
```

如果学生能完全理解这 20 行：

> *就已经理解了 Decoder-only LLM 预训练最核心的机制。*

MiniMind 剩余代码主要是在解决：

- *更现代的 Transformer 结构*
- *GPU 加速*
- *混合精度*
- *梯度累积*
- *多 GPU*

- *日志*
- *保存*
- *恢复*
- *MoE*

等工程问题。

---

86. 从概率角度再看一次源码

源码：

```
logits = self.lm_head(hidden_states)
```

代表模型参数化：

$$ P\theta(x{t+1}\mid x_{\le t}) $$

源码：

```
x = logits[..., :-1, :]
  y = labels[..., 1:]
```

代表：

$$ (x{\le t}, x{t+1}) $$

监督对。

源码：

```
F.cross_entropy(x, y)
```

代表：

$$ -\log P\theta(x{t+1}\mid x_{\le t}) $$

全数据训练就是：

$$ \boxed{ \min_\theta -\sum_{\text{documents}} \sum_t \log P_\theta(x_{t+1} \mid x_{\le t}) } $$

整个 Pretraining 最终就是这一个式子。

---

87. 从“信息压缩”的角度理解语言模型

一个好语言模型能给真实文本更高概率。

例如：

> *中国的首都是北京*

好模型可能：

> *P(北京 | 中国的首都是) = 0.8*

差模型：

> *P(北京 | 中国的首都是) = 0.0001*

Negative log probability：

好模型：

$$ -\log0.8 $$

很小。

差模型：

$$ -\log0.0001 $$

很大。

因此最小化 cross entropy，也可以理解成：

> *让模型越来越能够“压缩”真实语言。*

越可预测，编码真实文本所需的信息量越低。

---

88. Perplexity（PPL）

如果平均 token loss：

$$ L $$

定义：

$$ \boxed{ PPL=e^L } $$

例如：

$$ L=2 $$

则：

$$ PPL=e^2\approx7.39 $$

直观上可粗略理解成：

> 模型平均每一步像在约 7.39 个等可能候选中犹豫。

PPL 越低通常表示语言建模能力越好。

但注意：

> 不同 tokenizer 下 PPL 不宜简单横向比较。

MiniMind README 也专门指出 tokenizer 会影响按 token 统计的 PPL，跨 tokenizer 时 BPB 往往更公平。

---

89. 为什么 MiniMind 词表只有 6400？

对于小模型：

Embedding / LM Head 参数：

$$ V\times H $$

如果词表从：

$$ 6400 $$

扩大到：

$$ 128000 $$

hidden=768 时：

$$ 128000\times768 = 98.3M $$

光 embedding 就比整个 MiniMind 还大。

所以小模型必须认真权衡：

```
更大词表
  ↔
  更短 token 序列
  ↔
  更大的 embedding 参数
```

MiniMind 选择 6400 是一种小模型体积优先设计。

---

90. 小词表的代价

词表小意味着：

> 一个中文词、英文单词可能被拆成更多 token。

例如某个长英文词：

```
transformation
```

大 tokenizer 可能：

```
transformation
```

一个 token。

小 tokenizer 可能：

```
trans
  form
  ation
```

三个 token。

因此：

- 序列变长
- Attention 计算变多
- PPL 的 token 单位发生变化

这也是为什么 tokenizer 是 LLM 系统设计的重要一部分。

---

91. 最大序列长度 340 是什么意思？

训练参数：

```
--max_seq_len 340
```

表示当前 PretrainDataset 每条样本最终固定为：

$$ 340 $$

个 token。

如果原文短：

> PAD 到 340。

如果原文长：

> 截断到正文最多 338，再加 BOS/EOS。

注意这和模型配置：

```
max_position_embeddings = 32768
```

不是一回事。

---

92. `max_seq_len` 和 `max_position_embeddings`

`max_seq_len=340`

训练数据当前实际喂入的长度。

`max_position_embeddings=32768`

模型 RoPE 预计算支持的位置范围。

所以模型结构理论位置范围很长，但当前默认快速预训练只用 340 长度。

不能简单认为：

> “训练长度 340，因此模型已经充分学会处理 32768 token。”

长上下文能力仍然受到训练长度分布影响。

---

93. Flash Attention 在这里真正是什么？

MiniMind 判断：

```
self.flash = (
      hasattr(
          torch.nn.functional,
          'scaled_dot_product_attention'
      )
      and config.flash_attn
  )
```

满足条件时：

```
F.scaled_dot_product_attention(...)
```

这里更准确的课堂表述是：

> *使用 PyTorch 的 SDPA 接口；后端在合适硬件和条件下可能选择 Flash Attention 等高效 kernel。*

不要简单讲成：

> *这行一定就是某一个固定 FlashAttention 库。*

MiniMind 没有依赖独立 `flash-attn` *包来实现核心 Attention。*

---

94. 为什么 Attention 的复杂度是 (O(L^2))？

Attention score：

$$ QK^\top $$

shape：

$$ [L,d]\times[d,L] \rightarrow [L,L] $$

所以每个 token 都和其他可见 token 计算关系。

复杂度：

$$ O(L^2d) $$

memory 也容易受到：

$$ L^2 $$

影响。

例如长度从：

$$ 340\rightarrow680 $$

score matrix 面积变为：

$$ 4\times $$

这就是长上下文成本高的重要原因。

---

95. 预训练时到底在更新哪些参数？

默认：

```
from_weight = 'none'
```

即：

> *从随机初始化开始。*

而且：

```
optimizer = AdamW(model.parameters())
```

没有冻结任何层。

因此默认 pretrain：

> ***全部参数训练。***

包括：

- *token embedding*
- *Q projection*
- *K projection*
- *V projection*
- *output projection*
- *RMSNorm scale*
- *FFN gate/up/down*

- *LM Head（和 embedding tied）*

---

96. `from_weight` 是什么？

参数：

```
--from_weight none
```

如果不是 `none` *：*

```
weight_path =
  ../out/{from_weight}_{hidden_size}.pth
```

加载已有权重：

```
model.load_state_dict(
      weights,
      strict=False
  )
```

所以同一个 pretrain trainer 也能做：

> *基于已有权重继续训练。*

但默认 `none` *才是严格意义上的“从头预训练”。*

---

97. 随机初始化意味着什么？

训练开始前模型并不知道：

```
“北京”
```

和：

```
“中国”
```

有什么关系。

Embedding 也是随机的。

例如初始：

```
北京 → [0.01, -0.03, ...]
  中国 → [-0.02, 0.01, ...]
```

这些数一开始没有语义。

经过大量梯度更新后，相似上下文中的 token 会形成有结构的表示空间。

所以 embedding 的语义：

> *不是人预先写进去的，而是通过 next-token prediction 学出来的。*

---

98. 为什么 Pretrain 要大量数据？

假设只训练一句：

> *北京是中国的首都*

模型完全可以死记硬背。

但泛化不了。

当语料达到大量不同文本时：

> *单纯记住每个完整句子越来越困难。*

模型必须学习可复用的统计结构：

- *语法*
- *实体关系*
- *模式*
- *语义*
- *推理结构*

这就是规模化数据的重要性。

---

99. 数据质量为什么重要？

训练目标不会判断：

> *这句话是真的还是假的。*

它只会学：

> *数据里什么 token 经常跟在什么上下文之后。*

如果训练数据大量包含：

- *错误事实*
- *垃圾重复*
- *乱码*
- *广告模板*
- *极度低质量合成数据*

模型就会学习这些分布。
所以：

$$ \boxed{ \text{Model Quality} \neq \text{Only Model Architecture} } $$

还高度取决于：

$$ \text{Data Quality} $$

MiniMind README 也强调主线数据经过清洗、去重、长度控制和格式统一。

---

100. 一个训练 Step 的“电影慢镜头”

假设 batch 中有：

```
32 条文本
  每条最多 340 token
```

Step 1：Dataset

每条文本：

```
text
  ↓
  tokenizer
  ↓
  BOS + ids + EOS
  ↓
  PAD
  ↓
  input_ids
  labels
```

Step 2：送 GPU

```
input_ids.to(cuda)
  labels.to(cuda)
```

Step 3：Embedding

```
[32,340]
  →
  [32,340,768]
```

Step 4：8 个 Transformer Block

每层：

```
RMSNorm
  →
  QKV
  →
  QK Norm
  →
  RoPE
  →
  Causal GQA
  →
  Residual
  →
  RMSNorm
  →
  SwiGLU
  →
  Residual
```

Step 5：LM Head

```
[32,340,768]
  →
  [32,340,6400]
```

Step 6：Shift

```
预测位置：
  0...338

  目标：
  1...339
```

Step 7：忽略 PAD

```
label = -100
```

不参与 loss。

Step 8：CrossEntropy

得到一个标量：

```
loss = 例如 6.83
```

Step 9：除以 8

因为梯度累积：

```
6.83 / 8
```

Step 10：Backward

所有参数计算梯度。

Step 11：积累 8 次

第 8 次后：

```
unscale
  clip
  AdamW.step
  zero_grad
```

Step 12：继续下一个 batch

不断重复。

---

101. 初始 loss 大约会是多少？

这是课堂上很好的提问。

如果模型完全随机，6400 个 token 大致均匀：

$$ P(correct)\approx\frac1{6400} $$

loss：

$$ -\log\frac1{6400} = \log6400 $$

约：

$$ \boxed{8.76} $$

所以一个完全随机、近似均匀的 6400 词表语言模型，CrossEntropy 常可粗略从 8~9 左右理解。

训练后如果 loss：

```
8.7 → 6 → 4 → 3 → ...
```

说明模型对正确 token 的概率在不断提高。

注意真实初始化、数据分布、weight tying 等因素会使实际初始值不一定恰好等于 8.76。

---

102. Loss 降低到底意味着什么？

Cross entropy：

$$ L=-\log p $$

粗略地：

$$ p\approx e^{-L} $$

如果一个单独位置：

```
loss = 8
```

对应正确 token 概率约：

$$ e^{-8}\approx0.000335 $$

如果：

```
loss = 2
```

约：

$$ e^{-2}\approx0.135 $$

真实平均 loss 不能直接等同于每个位置都具有相同概率，但这个例子能帮助学生理解：

> *loss 每下降一点，正确 token 的整体概率分布已经发生了明显改善。*

---

103. 为什么训练时不关心 argmax 对不对？

假设正确 token：

> 北京

两个模型：

A：

```
北京 0.30
  上海 0.29
```

B：

```
北京 0.90
  上海 0.02
```

两者 argmax 都正确。

但 B 显然更有信心。

Cross Entropy 会区分：

$$ -\log 0.3 $$

和：

$$ -\log 0.9 $$

因此训练语言模型不是仅仅优化：

```
accuracy
```

而是在拟合完整概率分布。

---

104. 为什么生成时还要 temperature / top-p，而训练时不用？

训练目标：

> 对真实 token 最大化概率。

因此直接对完整 softmax 分布做 CrossEntropy。

生成时则要从：

$$ P(x_{t+1}|x_{\le t}) $$

中选择一个 token。

可以：

```
argmax
  sampling
  temperature
  top-k
  top-p
```

所以：

> *训练解决“学概率分布”。*

> *decoding 解决“怎么从概率分布选择 token”。*

两者不要混为一谈。

---

105. MiniMind 当前代码的几个教学级工程细节

细节 1：默认 BF16，不用 GradScaler

源码明确：

```
GradScaler(
      enabled=(dtype == 'float16')
  )
```

细节 2：无 warmup

`get_lr()` *是纯 cosine decay 到 10%。*

细节 3：PAD label = -100

利用 PyTorch：

```
ignore_index=-100
```

细节 4：Dataset 不创建 attention_mask

对于右侧 padding，Pretrain forward 没传显式 attention mask。

但因为：

- loss 对 padding target 做了 ignore
- causal mask 限制当前位置只能看左侧
- PAD 位于序列右侧

所以真实正文 token 不会看到未来的右侧 PAD。
PAD 自己产生的 hidden state 虽然会被计算，但其 target loss 被忽略。

这是一种简化实现。

---

106. 关于“没有 attention_mask”的更严谨解释

例如：

```
BOS 我 爱 EOS PAD PAD
```

当位置：

```
“我”
```

做 causal attention 时，它只能看：

```
BOS
  我
```

看不到右侧 PAD。

因此右 padding 不会污染正文 token 的表示。

PAD 位置本身可以看前文，但：

```
它们的 labels = -100
```

所以不贡献语言模型 loss。

因此在这种：

> ***全部 padding 都在右侧 + causal decoder***

的简单 pretrain 场景中，可以不显式传 padding attention mask。

---

107. 但什么时候 attention_mask 很重要？

例如：

- *左 padding*
- *packed sequences*
- *多段样本拼接但不希望跨样本 attention*
- *更复杂 batch packing*
- *推理场景*
- *不规则 padding*

这时不能简单依赖 causal mask。
所以不要把 MiniMind 这里的简化结论推广成：

> *Decoder LLM 永远不需要 attention mask。*

---

108. 最后一个不满 accumulation_steps 的 batch

MiniMind 训练 epoch 结束时：

```
if last_step % accumulation_steps != 0:
      ...
      scaler.step(optimizer)
```

所以不会丢掉最后残余的梯度。

但这里有一个很细的实现特点：

每个 micro-batch 都提前：

```
loss /= accumulation_steps
```

即便最后只剩：

```
3 个 micro-batch
```

仍然每个除以 8。

所以最后那个不完整 accumulation group 的梯度幅度会比“按实际 3 个 batch 平均”更小。

这是为了代码简洁可以接受的实现细节，但如果做严格大规模训练，可以对最后剩余 micro-batch 数重新归一化。

---

109. DDP + Gradient Accumulation 的通信细节

MiniMind 在梯度累积期间没有显式使用：

```
model.no_sync()
```

因此 DDP 通常会在每个 backward 都进行梯度同步。

数学结果仍然可以正确累积，但通信效率不是最极致。

更典型的优化做法：

```
前 7 个 accumulation micro-batch：
  no_sync()

  第 8 个：
  正常 backward + all-reduce
```

这样能减少多卡通信。

MiniMind 的取舍很明显：

> 更重视代码清晰、易学，而不是把训练系统优化写得极其复杂。

---

110. `torch.compile`

参数：

```
--use_compile 1
```

则：

```
model = torch.compile(model)
```

PyTorch 会尝试：

- *捕获计算图*
- *编译 / 融合部分算子*
- *减少 Python overhead*

可能加速训练。
但：

- *首次编译有开销*
- *不同 PyTorch / GPU 环境收益不同*
- *动态 shape 等情况可能影响效果*

所以默认关闭。

---

111. Random Seed

MiniMind：

```
setup_seed(args.seed + rank)
```

不同 DDP rank：

> *使用不同 seed。*

每个 epoch：

```
setup_seed(args.seed + epoch)
```

确保：

> *每轮 shuffle 变化，但仍尽量可复现。*

并设置：

```
torch.backends.cudnn.deterministic = True
  torch.backends.cudnn.benchmark = False
```

这是偏可复现性的选择。

---

112. 什么叫“训练一个 Base Model”？

完成 pretrain 后得到：

```
out/pretrain_768.pth
```

这个模型的能力应该理解为：

$$ P_\theta(\text{next token}\mid\text{prefix}) $$

它已经有：

- *语言建模能力*
- *文本续写能力*
- *一定知识*
- *一定代码/逻辑模式能力*

但还没有经过充分的：

```
instruction alignment
```

所以通常下一步：

```
python train_full_sft.py
```

把它变成更会：

```
理解用户问题
  按照 assistant 角色回答
  遵守格式
```

的模型。

---

113. Pretraining 学到什么，SFT 又学到什么？

可以给学生画一个二维类比：

> *Pretraining：*
> *“你读过多少书？”*
>
> *SFT：*
> *“你是否学会如何参加问答考试？”*
>
> *Alignment：*
> *“你的回答是否符合人类偏好与行为约束？”*

更严谨地：

> *Pretrain：*
> *建模自然/人工文本总体分布*
>
> *SFT：*
> *条件化到指令-回答分布*
>
> *DPO / PPO / GRPO：*
> *进一步优化偏好/奖励目标*

---

114. 课堂上最容易出现的 12 个误区

误区 1

“Pretrain 的 label 是人工标的。”
错。

> *label 就是文本下一个 token，自监督生成。*

误区 2

“input_ids == labels，所以模型在抄答案。”
错。

> *模型内部 shift 一位。*

误区 3

“模型每次只预测最后一个 token。”
错。

> *训练时一次 forward 并行预测几乎所有位置。*

误区 4

“训练时可以看到完整句子，所以未来 token 泄漏了。”
错。

> *causal mask 阻止每个位置看到未来。*

误区 5

“CrossEntropy 输入的是 probability。”
错。

> *PyTorch `F.cross_entropy` 输入 logits。*

误区 6

“PAD 也应该预测。”
错。

> *MiniMind 用 `-100` ignore。*

误区 7

“Embedding 是预先就有语义的。”
错。

> *从随机参数经过预训练学出语义。*

误区 8

“Pretrain 后就一定会像 ChatGPT 一样回答。”
错。

> *Base Model 与 Instruction Model 不同。*

误区 9

“RoPE 就是把 position embedding 加到 token embedding。”
错。

> *RoPE 主要旋转 Q/K。*

误区 10

“MiniMind 是普通 8-head MHA。”
不准确。

> *Q=8 heads，KV=4 heads，是 GQA。*

误区 11

“batch_size=32 就是实际更新 batch size。”
不完整。

> *accumulation=8，所以单卡有效 batch 约 256。*

误区 12

“模型是在记一句句答案。”
不完整。

> *参数通过共享统计结构来压缩和建模文本分布，记忆只是其中可能的一部分现象。*

---

115. 可以在课堂现场手算的一道题

给定：

```
input_ids =
  [BOS, A, B, C, EOS, PAD]

  labels =
  [BOS, A, B, C, EOS, -100]
```

问：

> 最终有几个 token target 会贡献 loss？

模型 shift：

```
logits：
  位置 0,1,2,3,4

  labels：
  A,B,C,EOS,-100
```

所以真正参与：

```
A
  B
  C
  EOS
```

共：

$$ \boxed{4} $$

个 target。

---

116. 第二道课堂题：Shape

设：

```
batch = 4
  seq_len = 10
  hidden = 128
  vocab = 1000
```

问：

LM Head 前：

$$ [4,10,128] $$

LM Head 后：

$$ \boxed{[4,10,1000]} $$

shift 后 logits：

$$ \boxed{[4,9,1000]} $$

target：

$$ \boxed{[4,9]} $$

flatten：

$$ \boxed{[36,1000]} $$

和：

$$ \boxed{[36]} $$

所以一次 CE 是 36 道 1000 分类题。

---

117. 第三道课堂题：随机模型理论 loss

词表：

$$ V=6400 $$

如果均匀预测：

$$ P=\frac1{6400} $$

CE：

$$ L=\log6400 $$

约：

$$ \boxed{8.76} $$

因此如果学生看到初始 loss 大约 8~9：

> 不要马上认为训练坏了。

先看词表大小和初始分布。

---

118. 第四道课堂题：有效 Batch Size

MiniMind：

```
batch_size = 32
  accumulation_steps = 8
  GPU = 4
```

全局有效 batch：

$$ 32\times8\times4 = \boxed{1024} $$

如果序列长度 340，理论输入 token / optimizer update：

$$ 1024\times340 = \boxed{348160} $$

---

119. 第五道课堂题：为什么 EOS 也要训练？

如果不训练 EOS：

> 模型只会学“接下来写什么”，却没有明确学习“什么时候结束”。

加入 EOS 后：

> 文章最后一个正文 token
> → EOS

模型学习：

$$ P(EOS\mid\text{完整上下文}) $$

生成时才能合理停止。

---

120. MiniMind Pretrain 的一句话源码总结

如果把所有工程细节去掉，它做的事情就是：

$$ \boxed{ \text{Text} \rightarrow \text{Tokenizer} \rightarrow x1,\ldots,xT \rightarrow \text{Transformer} \rightarrow P(x{t+1}|x{\le t}) \rightarrow CE \rightarrow \text{Backprop} } $$

其中训练目标：

$$ \boxed{ \min_\theta -\sum_t \log P_\theta(x_{t+1} \mid x_{\le t}) } $$

这就是 MiniMind Pretraining 的数学本质。

---

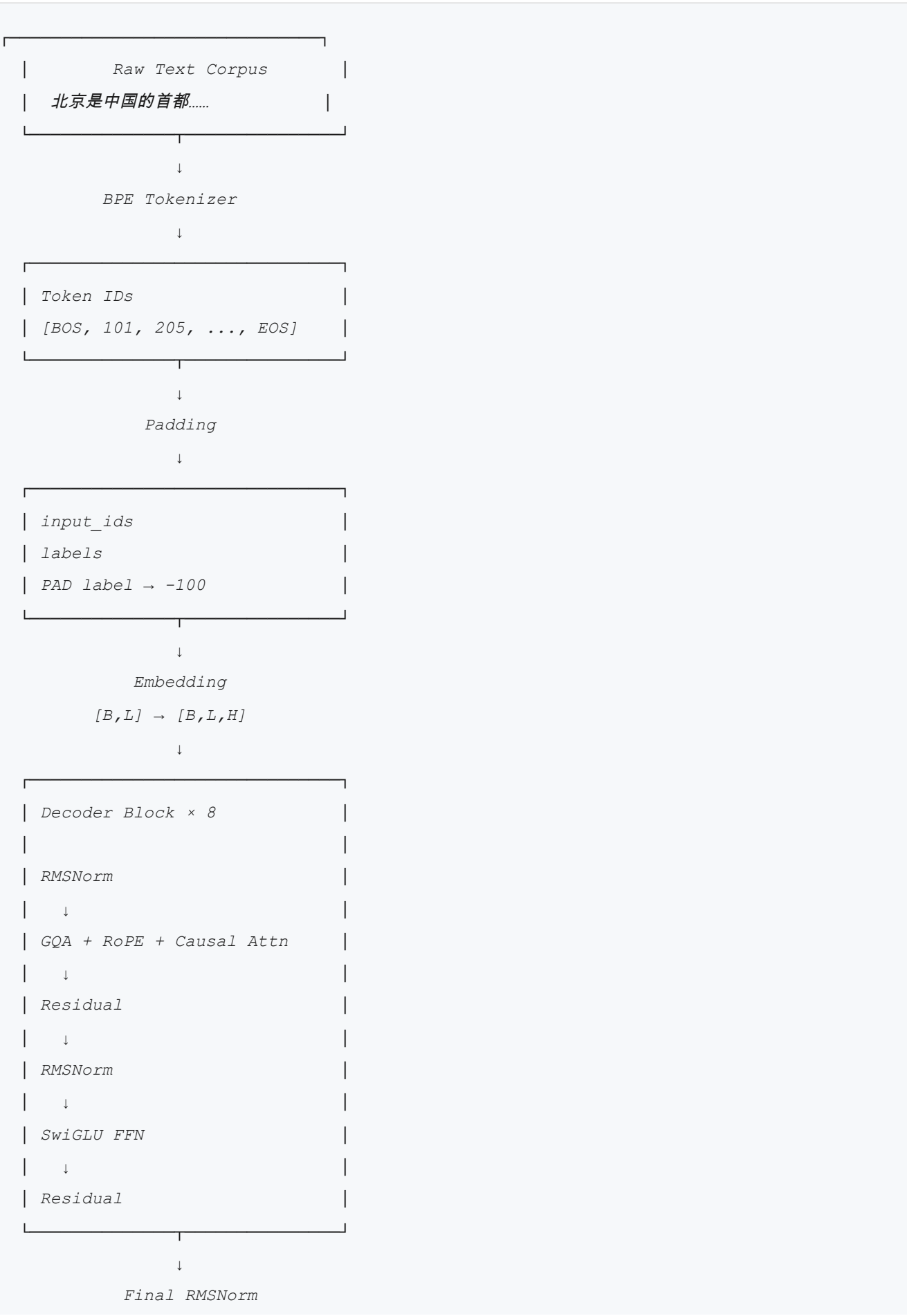
121. 对应源码速查表

概念	MiniMind 文件	关键位置
Pretrain 主程序	`trainer/train_pretrain.py`	`train_epoch()`、`__main__`
原始文本读取	`dataset/lm_dataset.py`	`PretrainDataset.__init__`
Tokenize	`dataset/lm_dataset.py`	`PretrainDataset.__getitem__`
BOS/EOS	`dataset/lm_dataset.py`	`tokens = [bos] + ... + [eos]`
Padding	`dataset/lm_dataset.py`	`input_ids = tokens + [pad] * ...`
Loss Mask	`dataset/lm_dataset.py`	`labels[PAD] = -100`
模型配置	`model/model_minimind.py`	`MiniMindConfig`
RMSNorm	`model/model_minimind.py`	`RMSNorm`
RoPE	`model/model_minimind.py`	`precompute_freqs_cis` / `apply_rotary_pos_emb`
GQA	`model/model_minimind.py`	`Attention` / `repeat_kv`
causal mask	`model/model_minimind.py`	`scaled_dot_product_attention(... is_causal=True)` 或上三角 mask
SwiGLU	`model/model_minimind.py`	`FeedForward.forward`
Transformer block	`model/model_minimind.py`	`MiniMindBlock`
Embedding	`model/model_minimind.py`	`MiniMindModel.embed_tokens`
LM Head	`model/model_minimind.py`	`MiniMindForCausalLM.lm_head`
Shift labels	`model/model_minimind.py`	`logits[..., :-1]` / `labels[..., 1:]`
CrossEntropy	`model/model_minimind.py`	`F.cross_entropy`
LR schedule	`trainer/trainer_utils.py`	`get_lr`
AdamW	`trainer/train_pretrain.py`	`optim.AdamW`

概念	MiniMind 文件	关键位置
Gradient accumulation	`trainer/train_pretrain.py`	`loss /= accumulation_steps`
Gradient clipping	`trainer/train_pretrain.py`	`clip_grad_norm_`
Mixed Precision	`trainer/train_pretrain.py`	`autocast` / `GradScaler`
DDP	`trainer/train_pretrain.py`	`DistributedDataParallel`
Checkpoint	`trainer/trainer_utils.py`	`lm_checkpoint`

---

122. 最后给学生的一张“总图”



```
                ↓
             LM Head
        [B,L,H] → [B,L,V]
                ↓
              Logits
                ↓
┌──────────────────────────────┐
│ Shift                        │
│ logits[:, :-1]               │
│ labels[:, 1:]                │
└──────────────┬───────────────┘
                ↓
          Cross Entropy
                ↓
         LM Loss (+ MoE Aux)
                ↓
            Backward
                ↓
       Gradient Accumulation
                ↓
         Gradient Clipping
                ↓
              AdamW
                ↓
       更新 MiniMind 参数
                ↓
    更准确预测下一个 Token
```

---

123. 教师讲授时建议抓住的三个核心公式

如果整堂课最后学生只能记住三个公式，应当是：

公式一：自回归分解

$$ \boxed{ P(x_{1:T}) = \prod_{t=1}^{T} P(xt|x{

公式二：预训练 Loss

$$ \boxed{ \mathcal L = -\frac1N \sum_t \log P_\theta(x_{t+1} \mid x_{\le t}) } $$

公式三：Causal Self-Attention

$$ \boxed{ \mathrm{Attention}(Q,K,V) = \mathrm{softmax} \left(\frac{QK^\top}{\sqrt{d_k}} + M_{\text{causal}} \right)V } $$

这三个公式分别回答：

> 语言概率如何分解？
>
> 模型到底在优化什么？
>
> 为什么训练时不能看到未来？

---

124. 一句话收尾

MiniMind 的 Pretrain 并没有神秘的“知识注入”模块。

它只是把海量文本变成海量：

> 前缀 → 下一个 token

的预测任务，然后通过：

```
Causal Transformer
  +
  Cross Entropy
  +
  Backpropagation
  +
  AdamW
```

不断调整约 6400 万个参数。

当这件简单的事被重复足够多次、使用足够多高质量文本之后，模型参数中逐渐形成了对语言、知识和各种模式的压缩表示。

这就是大语言模型预训练最核心的机制。

附录 A：MiniMind 默认 Pretrain 参数

```
epochs                 = 2
  batch_size               = 32
  learning_rate            = 5e-4
  dtype                    = bfloat16
  num_workers              = 8
  accumulation_steps       = 8
  grad_clip                = 1.0
  log_interval             = 100
  save_interval            = 1000
  hidden_size              = 768
  num_hidden_layers        = 8
  max_seq_len              = 340
  use_moe                  = 0
  seed                     = 42
  from_weight              = none
  from_resume              = 0
  use_compile              = 0
  vocab_size               = 6400
  num_attention_heads      = 8
  num_key_value_heads      = 4
  head_dim                 = 96
  intermediate_size        = 2432
  max_position_embeddings = 32768
  rope_theta               = 1e6
  tie_word_embeddings      = True
```

附录 B：建议课堂讲解顺序

推荐不要一开始就打开 `train_pretrain.py` *从第一行念代码。*

更适合学生理解的顺序：

1. 什么是语言模型
2. 什么是 next-token prediction
3. 概率链式法则
4. NLL / Cross Entropy
5. 一个句子如何产生多个监督信号
6. causal mask
7. Tokenizer / BOS / EOS / PAD
8. MiniMind PretrainDataset
9. MiniMind forward 中的 shift
10. Embedding
11. Self-Attention
12. GQA
13. RoPE
14. RMSNorm
15. SwiGLU
16. LM Head
17. 完整 shape 流
18. backward
19. AdamW
20. gradient accumulation
21. mixed precision
22. gradient clipping
23. LR schedule
24. DDP
25. checkpoint
26. Pretrain vs SFT

这样学生是在每一步都知道：

> “下面这几行代码是为了实现我刚才学过的哪个数学概念。”

而不是被大量 PyTorch 工程代码淹没。